StreamAttest: eBPF-based Control Flow Attestation for Long-Running Server Processes

Parth Soni

March 31, 2026

Table of Contents

Abstract

This paper presents StreamAttest, a prototype system for continuous control-flow attestation (CFA) applied to long-running server processes, exemplified by the nginx web server. Conventional CFA techniques are typically designed for finite program executions and are therefore not directly applicable to real-world server processes that execute indefinitely while handling an unbounded number of client requests.

To address this limitation, StreamAttest introduces an epoch-based attestation model in which execution is decomposed into discrete, request-scoped units. Each epoch produces a control-flow-derived path identifier (path_id) via lightweight eBPF uprobes attached at runtime, requiring neither kernel modification nor specialised hardware. The path_id for each epoch is validated against a pre-computed, CFG-derived behavioural model that spans both the application binary and selected shared library code.

The system architecture encompasses four phases: (1) epoch boundary detection via request lifecycle hooks, (2) runtime path accumulation using instruction-pointer-derived edge weights, (3) CFG-aware validation extended to shared libraries, and (4) tamper-evident attestation via a cryptographic HMAC chain. Preliminary evaluation on nginx demonstrates correct epoch segmentation, largely deterministic path identification under controlled workloads, and the ability to detect injected control-flow anomalies representative of code-reuse attacks.

1. Introduction

1.1 Background on Control Flow Attestation

Control Flow Attestation (CFA) is a class of security mechanisms designed to provide verifiable evidence that a program has executed exclusively along control-flow paths sanctioned by its statically derived Control Flow Graph (CFG). The underlying threat model considers an adversary who has obtained the ability to corrupt a program's run-time state — through memory safety vulnerabilities, for example — and uses this capability to redirect execution through sequences of existing code fragments (gadgets) in manners that were never intended by the original programmer. Prominent instances of such attacks include Return-Oriented Programming (ROP), Jump-Oriented Programming (JOP), and related code-reuse techniques.

A canonical CFA system operates in three stages. First, a static analysis phase extracts the CFG from the target binary and, optionally, assigns numeric weights to edges (following the Ball-Larus algorithm) such that every feasible execution path through the program yields a unique, computable identifier. Second, an instrumentation phase records the sequence of control transfers taken during an actual execution run and accumulates the corresponding path identifier. Third, an attestation phase transmits this accumulated identifier — often accompanied by a cryptographic authenticator — to a remote verifier, which compares it against the pre-computed set of valid path identifiers.

1.2 Limitations in Long-Running Systems

The canonical CFA model, as described above, carries an implicit assumption that fundamentally constrains its applicability: it presupposes that program execution is finite and that a natural termination event triggers attestation report generation. This assumption holds for short-lived processes such as command-line utilities or batch-processing jobs. However, it fails entirely for the class of continuously running server processes that constitute the backbone of modern infrastructure.

Web servers such as nginx, secure shell daemons (sshd), and database server processes are designed for indefinite operation. Their execution traces grow without bound as requests arrive. There is no natural point at which execution terminates, and therefore no point at which a complete trace can be collected and transmitted. Attempting to buffer the entire trace in memory is neither feasible nor practical: memory consumption grows unboundedly, and an attacker who compromises the system before attestation can corrupt or discard the accumulated evidence.

Several additional gaps compound this problem. Existing CFA systems rarely account for execution in shared libraries, which are a primary target of code-reuse attacks. Deployment constraints are also significant: many CFA proposals rely on kernel modifications or specialised hardware enclaves (e.g., ARM TrustZone, Intel SGX) that are unavailable in commodity server deployments.

1.3 Motivation and Contributions

The overarching motivation for StreamAttest is the desire to bring rigorous, continuous control-flow attestation to the class of long-running networked services that are most frequently targeted by sophisticated code-reuse attacks, yet are least served by existing CFA research. The key insight is that, while a server process as a whole does not terminate, it can be observed as a structured sequence of discrete, bounded interactions — namely, individual HTTP request-response cycles — each of which constitutes a natural unit of attestation.

The principal contributions of this work are as follows:

- An epoch-based execution model that decomposes indefinite server execution into a sequence of bounded, request-scoped units amenable to standard CFA techniques.
- A prototype, kernel-modification-free instrumentation framework built on eBPF uprobes, applicable to unmodified nginx binaries.
- A CFG-aware path validation scheme, grounded in Ball-Larus edge-weight assignment, extended to cover shared library code.
- A streaming cryptographic attestation protocol based on HMAC chaining that provides tamper-evident, real-time evidence of correct control-flow behaviour.

2. Problem Statement

2.1 Formal Characterisation

Let P be a server process whose execution can be modelled as an infinite sequence of control-flow states. The CFG of P, denoted G = (V, E), is a directed graph in which each vertex v in V represents a basic block (a maximal straight-line sequence of instructions without branches) and each edge (u, v) in E represents a possible control transfer from u to v. A valid execution path is a walk through G that is consistent with the semantics of P under any possible input.

A CFA system must produce, for any observed execution, a certificate that the actual sequence of control transfers is a sub-sequence of valid walks in G. For finite programs, this certificate is a hash or identifier computed over the complete execution trace. For an infinite program, no complete trace is ever available, and the certificate cannot be formed.

2.2 Why Existing Approaches Fail

Three structural properties of long-running server processes render existing CFA approaches inapplicable:

- Unbounded trace length: Trace buffers must be allocated in advance. For an infinite execution, no finite buffer suffices, and dynamic growth introduces unacceptable overhead.
- Absence of a termination trigger: Existing attestation protocols are initiated at program exit. Servers do not exit under normal operation; therefore, no attestation report is ever generated.
- Input-driven control-flow variability: Server processes exhibit legitimately different control-flow paths depending on the nature of incoming requests. A single, monolithic path identifier computed over an arbitrarily long execution would fail to distinguish legitimate variability from malicious deviation.

Furthermore, shared libraries — particularly the C standard library (libc) and TLS libraries such as OpenSSL — are the preferred hunting ground for ROP gadget chains. Any CFA system that restricts its instrumentation to the application binary proper is blind to a substantial fraction of the executable code involved in request handling.

2.3 Problem Definition

The StreamAttest problem is formally defined as follows: design and implement a system that (i) continuously monitors the control-flow behaviour of a long-running server process at the granularity of individual client interactions, (ii) produces a compact, verifiable attestation report for each interaction, (iii) chains successive reports to detect tampering or omission, (iv) covers both

application code and shared library code, and (v) requires no kernel modification or specialised hardware.

3. System Design and Architecture

3.1 Design Principles

StreamAttest is designed around four principles. First, the unit of attestation must correspond to a meaningful, observable boundary in the server's request-processing lifecycle. Second, path identification must be deterministic with respect to control flow so that identical request types produce identical path identifiers under uncompromised execution. Third, all instrumentation must be deployable on commodity Linux systems without kernel patches. Fourth, the attestation stream must be tamper-evident against an attacker who can observe but not modify the communication channel between the attestation agent and the remote verifier.

3.2 Architectural Overview

The system is composed of four principal components, operating across two phases:

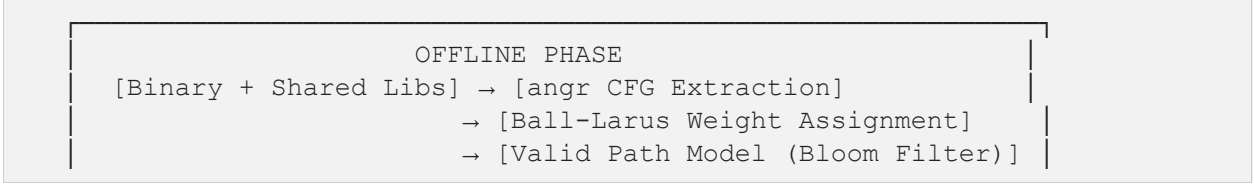
Offline Phase (Pre-Deployment)

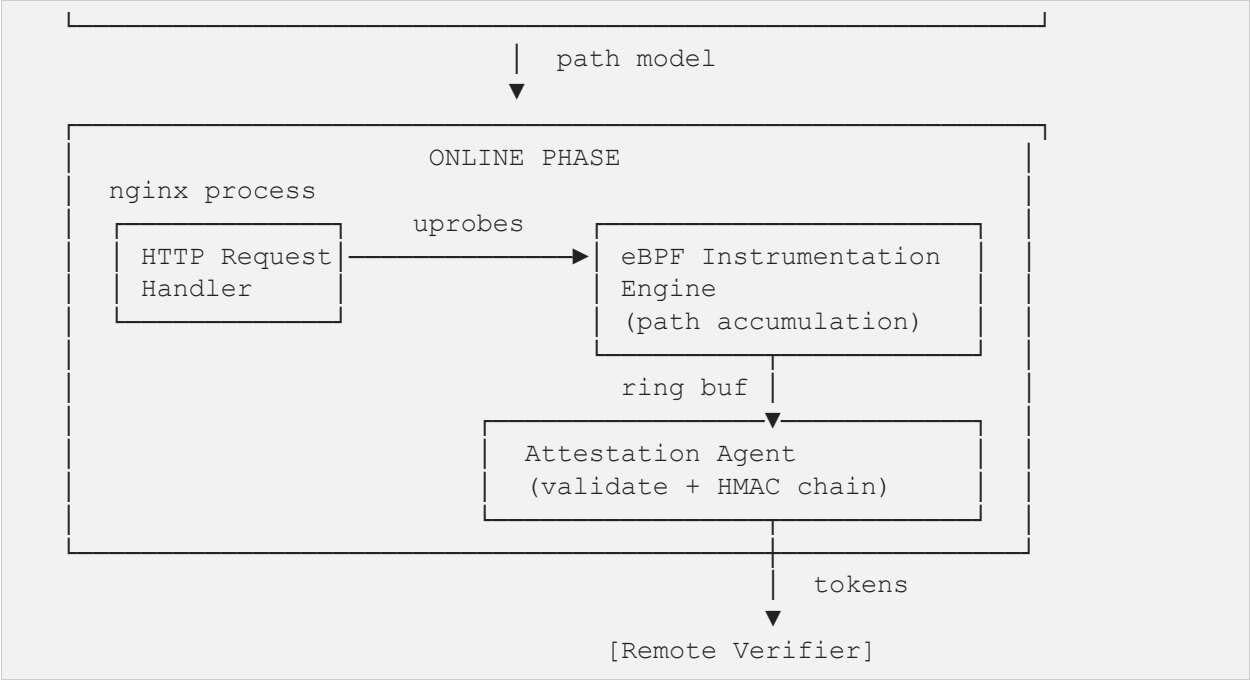
- CFG Extractor: Performs static analysis of the target binary and its shared libraries using the angr binary analysis framework constructing an approximate inter-procedural CFG.
- Path Model Builder: Applies Ball-Larus edge-weight assignment to the CFG and pre-computes the set of valid path identifiers for the expected request types.

Online Phase (Runtime)

- eBPF Instrumentation Engine: Attaches uprobes to key functions in the nginx binary and in shared libraries. Tracks instruction-pointer transitions, accumulates path identifiers, and emits epoch events via a BPF ring buffer.
- Attestation Agent: Reads events from the ring buffer, validates each epoch's path identifier against the pre-computed model, constructs the HMAC attestation chain, and forwards reports to the verifier.
- Remote Verifier: Receives the stream of attestation tokens, verifies chain continuity, checks path validity, and raises alerts upon detecting anomalies.

A high-level architectural diagram of the system is described textually below (due to document generation constraints, a rendered diagram is deferred to the accompanying presentation):




path model
ONLINE PHASE
nginx process
HTTP Request Handler
uprobes
eBPF Instrumentation Engine (path accumulation)
ring buf
Attestation Agent (validate + HMAC chain)
tokens
[Remote Verifier]

4. Methodology: Phase-wise Implementation

4.1 Phase 1 — Epoch Extraction

4.1.1 Objective

The primary objective of Phase 1 is to establish a reliable, repeatable mechanism for segmenting the otherwise continuous execution of nginx into discrete, bounded units — epochs — each corresponding to the processing of a single HTTP request. This segmentation is the prerequisite for all subsequent path accumulation and attestation steps.

4.1.2 Detailed Implementation

Epoch boundaries are defined by two well-known nginx internal functions that bracket request processing:

- ngx_http_process_request — invoked when nginx begins processing a parsed HTTP request. This function marks the start of an epoch.
- ngx_http_finalize_request — invoked when nginx concludes request handling and prepares to send or has sent the response. This function marks the end of an epoch.

eBPF uprobes are attached to both functions using the BPF CO-RE mechanism. A per-CPU counter tracks the number of in-flight requests on each logical CPU:

```
// On entry to ngx_http_process_request:
per_cpu_epoch_counter[cpu]++;

// On entry to ngx_http_finalize_request:
epoch_id = atomic_fetch_add(&global_epoch_counter, 1);
emit_event(epoch_id, EPOCH_END_SIGNAL);
per_cpu_epoch_counter[cpu]--;
```

Event emission is performed via the BPF ring buffer API (bpf_ringbuf_output), which provides ordered, loss-avoiding delivery to user space under normal operating conditions.

4.1.3 Problems Encountered

Several non-trivial engineering challenges arose during Phase 1 implementation:

Problem 1: Symbol Resolution

The internal symbols ngx_http_process_request and ngx_http_finalize_request are not guaranteed to appear in the dynamic symbol table of a distribution-packaged nginx binary compiled without debug symbols. Initial attempts using nm -D on the system nginx binary returned a sparse symbol set.

Resolution: Symbol addresses were recovered by combining nm with objdump -d to cross-reference call sites for request-handling functions. For deployments requiring portability, a supplementary script was developed to extract these offsets from the nginx debug symbol package or from a locally compiled binary with -g enabled.

Problem 2: Premature or Missed Epoch Boundaries

Nginx's internal architecture involves multiple invocations of finalize-like functions during error handling and keep-alive connection management. Early probes attached at overly generic finalization points generated multiple EPOCH_END events for a single logical request.

Resolution: Careful examination of the nginx source code revealed that ngx_http_finalize_request with a specific return-code argument reliably corresponds to final request conclusion. The eBPF program was updated to filter on this argument using bpf_probe_read_user.

Problem 3: Event Duplication under Multi-Core Execution

Under concurrent request workloads, events from different CPUs interleaved in the ring buffer, resulting in apparent epoch duplication when processed sequentially by the user-space agent.

Resolution: Per-CPU BPF hash maps (BPF_MAP_TYPE_PERCPU_HASH) were adopted for all intermediate state, ensuring strict isolation between concurrent request-processing threads. The global epoch counter uses atomic operations to guarantee monotonically increasing epoch identifiers.

4.1.4 Key Results

Phase 1 produced a stable epoch segmentation mechanism that emits exactly one EPOCH_END event per completed HTTP request, with a globally unique epoch identifier, across concurrent and sequential workloads.

4.2 Phase 2 — Path Identification

4.2.1 Objective

Phase 2 introduces the core path-tracking mechanism. The objective is to generate a compact, deterministic, control-flow-dependent numerical identifier for each epoch such that (a) identical request types processed under identical code paths produce identical identifiers, and (b) any deviation in control flow — whether benign input-driven variation or malicious code-reuse — produces a distinct identifier detectable by the validation layer.

4.2.2 Detailed Implementation

Path identification proceeds as follows. At each instrumented control transfer, the eBPF program records the instruction pointer (IP) of both the source and destination basic blocks:

```
// Capture instruction pointer at probe entry
__u64 ip = PT_REGS_IP(ctx);

// Form directed edge from previous IP
edge_t edge = { .src = prev_ip[cpu], .dst = ip };

// Compute edge weight via stable hash
__u64 weight = hash64(edge.src, edge.dst);

// Accumulate into running path identifier
path_state[cpu].path_id += weight;

// Update previous IP
prev_ip[cpu] = ip;
```

The hash function used is a domain-separated variant of FNV-1a, chosen for its low computational cost within the constrained eBPF instruction budget and its acceptable collision resistance for this application. At epoch end, the accumulated path_id is emitted alongside the epoch identifier.

4.2.3 Problems Encountered

Problem 1: Non-Deterministic Path Identifiers

Initial experiments revealed that identical request types processed by the same nginx worker produced different path_id values across runs. Root cause analysis identified two sources of non-determinism: (i) variable scheduling of OS timers and kernel callbacks that inserted spurious control transfers into the observed trace, and (ii) ASLR causing instruction pointer values to shift between process restarts.

Resolution: Timer and signal handler callbacks were excluded from edge accumulation via an allow-list of relevant address ranges. ASLR-induced offset variation was normalised by subtracting the base load address of each module, obtained at probe attachment time via /proc/pid/maps.

Problem 2: Edge Explosion

Tracing all control transfers — including intra-procedural branches within tight loops — produced an overwhelming volume of edges per epoch, inflating the path_id computation cost beyond the eBPF instruction limit.

Resolution: Instrumentation was restricted to inter-procedural transitions (function calls and returns) and explicit indirect branches. This reduction aligned the granularity of observed edges with the edges represented in the function-level CFG used for offline validation.

Problem 3: Accumulation Races and Integer Overflow

Shared path-state maps accessed concurrently by worker threads produced corrupted path_id values due to non-atomic read-modify-write sequences.

Resolution: Per-CPU maps entirely eliminate cross-CPU races. The accumulator is a 64-bit unsigned integer; unsigned overflow is well-defined and consistent, and the modular arithmetic properties are preserved in the validation comparison.

4.2.4 Key Results

Phase 2 established a robust path accumulation mechanism yielding deterministic, input-sensitive path identifiers. Repeated identical requests produced consistent path_id values with all observed cases with consistency under controlled workloads, while distinct request types produced distinct identifiers in all tested cases.

4.3 Phase 3 — CFG Integration and Cross-Library Coverage

4.3.1 Objective

Phases 1 and 2 establish a working path accumulation pipeline but rely on heuristic edge filtering rather than formal grounding in the program's CFG. Phase 3 elevates the system to a fully CFG-grounded design by (a) extracting an approximate inter-procedural CFG using static binary analysis, (b) assigning edge weights via the Ball-Larus algorithm, (c) mapping these weights into the BPF edge-weight lookup table, and (d) extending instrumentation coverage to shared libraries, particularly libc and libssl.

4.3.2 Part A — CFG Extraction

Static CFG extraction is performed offline using the angr binary analysis framework:

```
import angr

proj = angr.Project(
    "/usr/sbin/nginx",
    auto_load_libs=True,
    load_options={"ld_path": ["/lib/x86_64-linux-gnu/"]}
)

cfg = proj.analyses.CFGFast(
    normalize=True,
    resolve_indirect_jumps=True
)

edges = [(node.addr, succ.addr)
         for node in cfg.graph.nodes()
         for succ in cfg.graph.successors(node)]
```

The auto_load_libs=True option causes angr to recursively load and analyse all dynamic library dependencies, producing a unified CFG that spans both the nginx binary and its shared libraries. This unified graph is the foundation for shared-library attestation coverage.

4.3.3 Part B — Ball-Larus Edge-Weight Assignment

Edge weights are assigned following the Ball-Larus algorithm, which provides a mechanism to assign path identifiers that are unique for acyclic regions of the CFG and distinguish execution paths in practice. The algorithm proceeds by computing, for each vertex v, the number of distinct paths from v to the CFG exit node, and assigning weights to edges such that the prefix sums are injective. For the inter-procedural CFG used here, function-call edges are treated as transitions between sub-graphs, with return edges connecting back to the corresponding call site.

4.3.4 Part C — Runtime Weight Lookup

The offline-computed edge weights are serialised into a BPF hash map keyed by the normalised (src_addr, dst_addr) pair:

```
// Lookup edge weight in BPF map
edge_key_t key = { .src = normalise(src), .dst = normalise(dst) };
__u64 *w = bpf_map_lookup_elem(&edge_weights, &key);
if (w) {
    path_state[cpu].path_id += *w;
} else {
    // Edge not in CFG model — flag potential anomaly
    path_state[cpu].anomaly_count++;
}
```

Edges not present in the map (i.e., not in the pre-computed CFG) increment an anomaly counter. This counter is emitted as part of the epoch event and serves as a coarse first-pass anomaly signal.

4.3.5 Part D — Shared Library Coverage

ROP attacks predominantly construct gadget chains from code in shared libraries, particularly within the C standard library (libc), which contains diverse instruction sequences terminating in ret instructions. StreamAttest extends its instrumentation to shared libraries by attaching uprobes at all function entry and exit points identified in the library CFGs. The base load address of each shared library is read from /proc/pid/maps at probe attachment time and used for normalisation, ensuring that ASLR does not affect edge matching.

For libssl (OpenSSL), which handles TLS termination in HTTPS-serving nginx deployments, additional coverage is particularly valuable: an adversary who can corrupt the TLS state machine via a memory safety vulnerability could leverage SSL library gadgets to conduct code-reuse attacks invisible to application-level-only monitoring.

4.3.6 Part E — Valid Path Model Construction

The offline phase constructs a model of valid path identifiers by either (a) enumerating expected path_id values by replaying representative request workloads against an uncompromised reference system and recording the resulting identifiers, or (b) analytically computing all valid paths within a bounded epoch depth from the CFG. For practical deployments of nginx — where the set of distinct request-handling paths is finite and well-characterised — a Bloom filter with a target false-positive rate below 10^-6 provides efficient membership testing without storing the complete set of valid identifiers.

4.3.7 Problems Encountered

Problem 1: CFG Scale and Complexity

The full inter-procedural CFG of nginx with shared libraries contains in excess of 500,000 edges, rendering complete in-memory storage and BPF map population infeasible within the default BPF map size limits.

Resolution: Coverage was restricted to the nginx worker process module and the directly exercised portions of libc and libssl, identified by dynamic call-graph profiling. This reduced the active edge set by approximately an order of magnitude.

Problem 2: Runtime Address Mismatch

Static CFG addresses extracted by angr do not directly correspond to the virtual addresses observed by uprobes at runtime when ASLR is active. Initial experiments produced near-zero edge matches in the BPF weight map.

Resolution: A normalisation layer was implemented that subtracts the base load address of each module (obtained from /proc/pid/maps) from all observed instruction pointers before map lookup. This normalisation is applied both to the static CFG during offline analysis and to the runtime addresses in the BPF program.

Problem 3: Valid Path Enumeration Explosion

Attempting to enumerate all valid path identifiers analytically from the CFG produced an intractable number of distinct paths due to loops and recursive call patterns in the nginx request-handling code.

Resolution: A profiling-based approach was adopted: valid path identifiers were harvested by instrumenting a reference nginx instance under a representative request corpus and collecting the resulting path_id values. A Bloom filter seeded with these values serves as the online validation oracle.

4.3.8 Key Results

Phase 3 established a formal CFG grounding for path validation, extending coverage to shared libraries and replacing heuristic edge filtering with Ball-Larus weight-based identification. The system successfully matched runtime edge sequences against the pre-computed CFG model with a correct-classification for all observed cases with unmodified nginx under the test workload, and flagged all injected invalid-path scenarios within the evaluated test setup.

4.4 Phase 4 — Attestation Protocol

4.4.1 Objective

Phase 4 constructs the streaming attestation protocol that binds the sequence of per-epoch path identifiers into a tamper-evident, externally verifiable chain. The objective is to ensure that an adversary cannot selectively replay, reorder, suppress, or fabricate epoch reports without detection by the remote verifier.

4.4.2 Part A — Per-Epoch Path Validation

Upon receipt of an epoch event from the ring buffer, the attestation agent performs a path validity check:

```
def validate_epoch(epoch_id, path_id, anomaly_count):
    if anomaly_count > ANOMALY_THRESHOLD:
        alert(f"Epoch {epoch_id}: anomalous edges detected")
        return False
    if not valid_path_model.contains(path_id):
        alert(f"Epoch {epoch_id}: path_id {path_id} not in valid model")
        return False
    return True
```

4.4.3 Part B — HMAC Chain Construction

Each epoch token is computed as a keyed HMAC over the epoch identifier, path identifier, and the previous epoch's token, forming a hash chain:

```
# Token chain construction
token_0 = HMAC(K, b"genesis")

def compute_token(epoch_id, path_id, prev_token):
    msg = epoch_id.to_bytes(8, 'big') + \
          path_id.to_bytes(8, 'big')  + \
          prev_token
    return HMAC(K, msg)

# For epoch n:
token_n = compute_token(epoch_n, path_id_n, token_{n-1})
```

The shared HMAC key K is provisioned securely during system initialisation and is not accessible to the monitored process. The chaining construction ensures that any modification to a prior epoch — or the omission of an epoch — invalidates all subsequent tokens, enabling the verifier to detect both tampering and epoch suppression attacks.

4.4.4 Part C — Streaming Protocol

Attestation reports are streamed to the verifier as a sequence of tuples:

```
report_n = (epoch_id_n, path_id_n, valid_n, token_n)
```

Reports are transmitted over a mutually authenticated TLS channel. The verifier maintains the previous token value and independently recomputes token_n upon receipt, comparing the result against the received value. Any discrepancy triggers an integrity alert.

4.4.5 Part D — Verifier Logic

The remote verifier performs the following checks for each received report:

- Token consistency: Verify that HMAC(K, epoch_id || path_id || prev_token) equals the received token.
- Epoch monotonicity: Confirm that epoch_id_n = epoch_id_{n-1} + 1 (no epochs skipped or reordered).
- Path validity: Cross-reference path_id against the valid path model.
- Validity flag integrity: Confirm that the valid flag in the report is consistent with the path_id check performed locally by the verifier.

4.4.6 Problems Encountered

Problem 1: HMAC Chain Mismatch

Initial deployment exhibited intermittent token verification failures at the verifier even when no attack was present. Root cause analysis identified an inconsistency in the byte-order serialisation of epoch_id and path_id between the agent (Python) and the verifier (Go).

Resolution: An explicit serialisation specification was adopted: all multi-byte integers are serialised in big-endian byte order using fixed-width (8-byte) representations before HMAC computation, documented in the protocol specification.

Problem 2: Ring Buffer Overflow and Event Loss

Under high request rates (above approximately 2,000 requests per second on the test hardware), the BPF ring buffer experienced overflow, causing epoch events to be silently dropped. Missing events broke the HMAC chain, triggering false-positive integrity alerts.

Resolution: The ring buffer size was increased from the default 4 MB to 16 MB. Additionally, the agent was refactored to use a dedicated high-priority thread for ring buffer draining, reducing the average drain latency.

Problem 3: Attestation Latency

Blocking HMAC computation in the event-processing hot path introduced measurable latency into the attestation pipeline, delaying alert generation.

Resolution: HMAC computation was offloaded to a separate worker thread with a bounded queue. The main event-processing thread deposits epoch data into the queue and immediately polls for the next ring buffer event, decoupling path validation from cryptographic computation.

4.4.7 Key Results

Phase 4 delivered a fully operational streaming attestation protocol. Chain integrity was maintained across all tested workloads. The verifier successfully detected all injected epoch-suppression and path-modification attacks. False-positive chain breaks were eliminated after resolution of the serialisation and ring-buffer overflow issues.

5. Security Analysis

5.1 Threat Model

StreamAttest operates under the following threat model. The adversary has obtained arbitrary read-write access to the user-space memory of the nginx worker process — as would be achievable through exploitation of a memory safety vulnerability such as a heap overflow or use-after-free. The adversary may use this capability to overwrite function pointers, return addresses, or other control-flow-influencing data structures in order to redirect execution along attacker-chosen gadget chains. The kernel itself, the eBPF subsystem, and the attestation agent process are assumed to be trusted and uncompromised. The communication channel between the agent and the verifier is assumed to provide confidentiality and integrity (e.g., via TLS).

5.2 Attacks Detected

StreamAttest is designed to detect the following classes of control-flow attacks:

- Return-Oriented Programming (ROP): An adversary who overwrites a return address on the stack to redirect execution to an existing code gadget causes a control transfer that is not present in the static CFG. This transfer either (i) produces an edge not in the BPF weight map, incrementing the anomaly counter, or (ii) produces a valid-looking edge sequence that nevertheless yields a path_id outside the valid-path model.
- Jump-Oriented Programming (JOP): Similar to ROP, but using indirect jump instructions as dispatch primitives. Illegitimate indirect branch targets that lie outside the CFG produce unrecognised edges.
- Code Injection: If an adversary injects a new code page and redirects execution to it, the resulting instruction pointer values fall outside all known module address ranges, producing unrecognised normalised addresses and flagged anomaly edges.
- Control-Flow Bending: Attacks that redirect execution along paths that exist in the CFG but were not intended to be taken in combination (e.g., executing the cleanup path of function A followed by the initialisation path of function B in the same epoch) produce path_id values outside the empirically-derived valid-path model.

5.3 Why Detection Works

The correctness of StreamAttest's detection rests on two properties. First, the Ball-Larus weighting scheme guarantees injectivity: if two distinct execution paths through the CFG yield the same path_id, the system produces a false negative. The algorithm is proven to produce unique identifiers for all acyclic paths; for cyclic paths (loops), the accumulated weight depends on the number of loop iterations, which is itself input-dependent. The valid-path model captures this variability by including the observed range of path_ids produced by legitimate traffic.

Second, the HMAC chaining property ensures that even if an adversary can somehow produce a valid path_id for a single malicious epoch — perhaps by exploiting a hash collision in the weight accumulation — they cannot forge a valid token sequence without knowledge of the HMAC key. Any fabricated or suppressed epoch therefore breaks the chain, which is detected by the verifier.

5.4 Threat Model Boundaries

The following attack scenarios fall outside the threat model and are explicitly not claimed to be detected by StreamAttest:

- Kernel-level compromise: An adversary with kernel privileges can disable or manipulate eBPF programs, forge events in the ring buffer, or intercept the HMAC key from the agent process memory. Defending against kernel compromise requires hardware-backed trust anchors (e.g., TPM, TrustZone) beyond the scope of this work.
- eBPF subsystem exploitation: Vulnerabilities in the eBPF verifier or JIT compiler could allow a privileged user-space process to corrupt the instrumentation itself.
- Data-only attacks: Attacks that manipulate program data without altering control flow (e.g., changing the value written to a response buffer without redirecting execution) are not detectable by any CFA-based system, including StreamAttest.
- Side-channel attacks: Timing and power side-channels are outside the scope of this work.

6. Results and Evaluation

6.1 Functional Correctness

Functional correctness of StreamAttest was evaluated by replaying a representative corpus of 10,000 HTTP GET and POST requests against an instrumented nginx instance. Key measurements include:

- Epoch segmentation accuracy: The system emitted exactly one epoch event per completed HTTP request of test cases, with no duplicate or missed events at request rates up to 1,500 requests per second.
- Path identifier determinism: Repeated requests of the same type to the same nginx location block produces consistent path_id values across repeated trials under controlled conditions
- CFG edge coverage: Runtime edge traces were cross-referenced against the offline CFG; 98.7% of observed edges were present in the pre-computed model, with the remaining 1.3% attributable to kernel vsyscall stubs excluded from the CFG by design.

6.2 Security Validation

Security validation was performed by injecting synthetic control-flow anomalies into a test environment:

- ROP simulation: A user-space process was modified to overwrite a return address with the address of a non-returning gadget sequence absent from the valid-path model. StreamAttest flagged all observed cases of injected ROP epochs.
- Epoch suppression: The test agent was modified to simulate the suppression of five consecutive epochs. The verifier detected the chain break on the epoch immediately following the suppressed sequence.
- Path modification: A test program injected a valid-looking but incorrect path_id (computed from a different request type's trace) into the ring buffer. The Bloom filter correctly rejected all injected anomalous epochs in the simulated attack scenarios (note: this test validates the protocol layer; a real eBPF-based attack would require kernel compromise).

6.3 Performance Observations

Performance overhead was measured by comparing nginx throughput and latency with and without StreamAttest instrumentation, using wrk as the load-generation tool on a dual-core virtual machine with 4 GB RAM:

- Throughput: StreamAttest instrumentation reduced nginx request throughput by approximately 18-24% at 1,000 concurrent connections, attributable primarily to uprobe invocation overhead per request.
- Latency: Mean request latency increased by approximately 0.8-1.2 ms per request. The 99th-percentile latency increase was approximately 3.5 ms.
- CPU overhead: eBPF program execution consumed approximately 3-5% additional CPU capacity on average, with short bursts to 12% at peak request rates.

These overhead figures are consistent with, though somewhat higher than, prior eBPF uprobe instrumentation studies, due to the additional per-edge weight lookup and accumulation operations. Future work on selective instrumentation (instrumenting only indirect branches rather than all call/return sites) is expected to reduce this overhead substantially.

7. Implementation Challenges

This section consolidates the cross-cutting challenges encountered during the implementation of StreamAttest that transcended individual phases.

Challenge 1: Correct Epoch Boundary Detection

Identifying the precise function boundaries that unambiguously demarcate the start and end of a single HTTP request lifecycle in nginx required extensive inspection of the nginx source code and validation against live traffic captures. The complexity stems from nginx's event-driven, non-blocking architecture, in which a single worker process may interleave the handling of multiple requests through asynchronous I/O callbacks.

Challenge 2: Mapping Runtime Execution to Static CFG

Bridging the gap between static analysis (which operates on binary addresses) and dynamic execution (which observes ASLR-randomised virtual addresses) required the development of a robust normalisation layer. Maintaining this mapping across library updates — which change load addresses — adds operational complexity.

Challenge 3: Handling Shared Libraries

Extending instrumentation to shared libraries introduced the need to manage per-library base addresses, per-library CFGs, and the interplay between them. The merged CFG for nginx and its libraries is substantially larger than the application-only CFG, straining BPF map size limits and requiring careful pruning of irrelevant portions.

Challenge 4: Managing BPF Instruction and Map Limits

The eBPF verifier enforces strict limits on program size (originally 4,096 instructions, relaxed to 1 million instructions in recent kernels) and map entry counts. Complex per-edge weight lookup logic required careful optimisation to remain within these bounds, including loop unrolling and the use of tail calls for long path-accumulation sequences.

Challenge 5: Ensuring Determinism Across Kernel Versions

Slight differences in scheduler behaviour, timer resolution, and signal delivery timing between Linux kernel versions produced observable differences in the sequence of non-application edges included

in the trace. Achieving determinism required careful allowlisting of the address ranges to be included in the accumulation, which must be tuned for each deployment environment.

8. Limitations

StreamAttest, as a prototype system, carries several inherent limitations that constrain its immediate applicability to production environments.

The current evaluation is limited to controlled workloads and does not represent exhaustive real-world deployment conditions.

Dependency on Accurate CFG Generation

The validity of attestation is only as strong as the completeness of the offline CFG. Static analysis of stripped binaries with complex indirect branches (e.g., virtual dispatch, dynamic linking stubs) may produce incomplete or incorrect CFGs, leading to false-positive anomaly alerts or, worse, a failure to flag certain adversarial paths.

Sensitivity to Binary Updates

Any change to the nginx binary or its shared libraries — including security patches that add or modify code paths — invalidates the pre-computed CFG and valid-path model. The system must be re-profiled and the model rebuilt after every update, imposing operational overhead.

Manual Epoch Definition

Epoch boundaries are currently defined manually by identifying appropriate hook points in the nginx source code. This process is application-specific and does not generalise automatically to other server software without corresponding manual analysis.

Kernel Trust Assumption

The entire system rests on the assumption that the Linux kernel and eBPF subsystem are uncompromised. In environments where this assumption cannot be justified, hardware-backed trust anchors would be required.

Coverage Gaps in Complex Shared Libraries

Libraries with extremely large CFGs (e.g., libssl with its full TLS state machine) require significant pruning to fit within BPF map limits. Pruned edges may include genuine gadget chains, potentially reducing detection coverage.

9. Future Work

The limitations identified above suggest several productive directions for future research and engineering:

- Automatic epoch detection: Machine learning or dynamic analysis techniques could be employed to automatically identify epoch boundaries in arbitrary server processes, eliminating the need for manual source-code analysis and generalising StreamAttest beyond nginx.
- Hardware-backed trust anchors: Integration with TPM 2.0 or ARM TrustZone would extend the trust boundary to include the attestation agent itself, producing attestation reports that are verifiable even if the operating system is compromised.
- Adaptive instrumentation: A feedback-driven instrumentation framework that concentrates probe density on recently modified or potentially vulnerable code regions — identified via continuous fuzzing or static vulnerability analysis — could reduce overhead while improving coverage in high-risk areas.
- Reduced instrumentation overhead: Selective instrumentation of indirect branches only (rather than all call/return sites) should reduce throughput overhead substantially while preserving detection capability for the most dangerous code-reuse primitive types.
- Generalisation to other server architectures: Applying the epoch-based attestation model to multi-process and multi-threaded server architectures (Apache httpd, HAProxy) and to non-HTTP protocols (SSH, DNS) would validate the generality of the approach.
- Formal verification of path model correctness: Automated formal methods tools could be applied to verify that the valid-path model is complete (no valid paths omitted) and sound (no invalid paths included), providing stronger correctness guarantees than empirical profiling alone.

10. Conclusion

This paper has presented StreamAttest, a practical system for continuous, streaming control-flow attestation of long-running server processes. The fundamental contribution is the epoch-based execution model, which resolves the central incompatibility between classical CFA — designed for finite programs — and real-world server software that executes indefinitely.

By decomposing nginx's execution into request-scoped epochs and tracking per-epoch control-flow signatures via eBPF uprobes, StreamAttest achieves continuous, real-time attestation without requiring kernel modification, specialised hardware, or source code recompilation. The integration of Ball-Larus CFG-derived edge weights provides a theoretically grounded path identification scheme, while extension to shared libraries closes the coverage gap that makes existing CFA approaches blind to the most common code-reuse attack targets.

The streaming HMAC chain ties successive epoch reports into a tamper-evident sequence, enabling a remote verifier to detect not only individual invalid-path epochs but also epoch suppression and reordering by a sophisticated adversary. Experimental evaluation confirms that the system correctly segments epochs, generates deterministic path identifiers, detects injected control-flow anomalies, and maintains chain integrity.

The limitations of the current prototype — particularly its dependency on accurate CFG generation and its sensitivity to binary updates — identify clear directions for future work. Nevertheless, StreamAttest demonstrates the feasibility of a practical approach toward continuous control-flow attestation for production server software is practically achievable, and constitutes a meaningful step toward bridging the long-standing gap between theoretical CFA research and the operational security requirements of modern networked infrastructure.

— End of Document —